

2026 Spring

Course ID: 1142-1765

Advanced Programming



09-Rust and CPE 1



Advanced Programming

Python · C++ · Rust

Data Structures & Algorithms



Python



C++

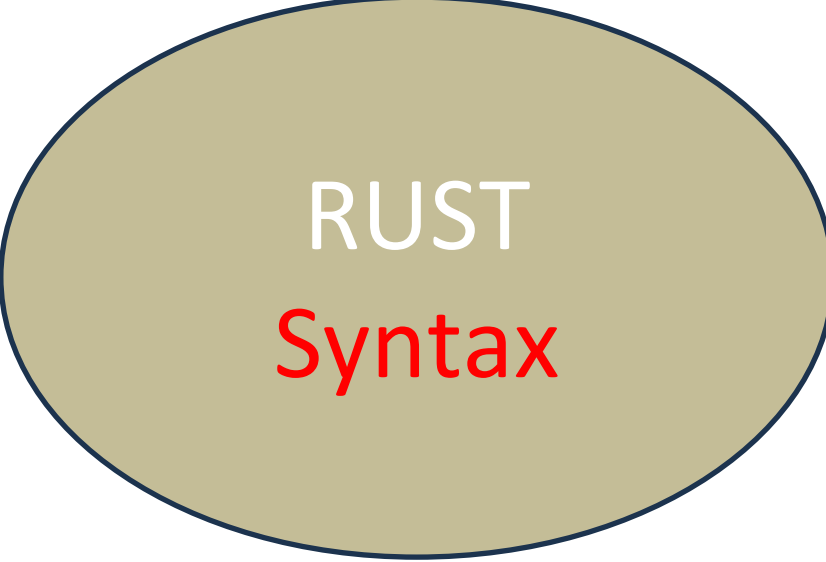


Rust

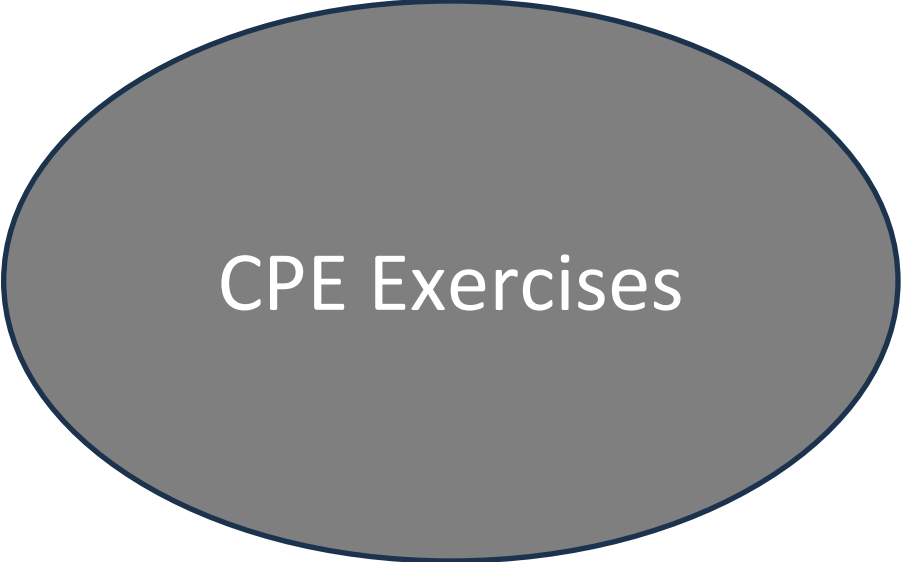
Part-1



Course Activities
Discuss midterm
presentation



RUST
Syntax



CPE Exercises

Group-1

411330111 安佐駟 Tserendash.G
411330145 安佳卡 Nomunbileg Azjargal
411330200 安素恩 Lkhamsuren Ulziikhishig
411426140 庫爾特 KURT CZEDRICK SASUMAN LANDICHO
411419274 尔汀巴塔 ERDENEBAATAR AMARBAYAN
411419282 涂尔巴亞 BATJARGAL TURBAYAR

Group-2

411419313 杰 登 ROUSE JAYDEN ARIEL ANTYONNE
411419321 馬坎亞 Xolani Ignicious Makhanya
411425194 包格倫 GLENN PASCUA BAUTISTA JR.
411425217 艾德華 EDWARD ASIA CASTILLO
411426077 婕 登 MUHLE JADEN VILANE
411416959 歐匠伯 Abraham

Group-3

411412141 洪健智
411200594 黃宇碩
411211456 林靖翔
411211650 陳冠中
411120320 梁偉航
E11410470 溫依薇 WADDINGTON EVEY

Group-4

411314717 李研熙 Gloria Ananda Justine
411416878 鄭鎧文 CALVIN THEODORIC LAVENCIO
411416886 蔡美彥 MANDY JANNYCIA MAHIDIN
411416991 許原盛 Yansen Ladova
411419355 吳加寶 WILLYANTO SUDINO

Group-5

411425178 鄭永全 YOSEF CHRISTIAN
411425225 吉 曼 AHMAD SOLEHUDIN AL AZIZ
411419305 譚碧雄 CHARLES SEPTIAN PRATAMA
411419258 雨海南 VU HAI NAM
E11420600 哈里哈蘭 UDAYA SANKAR HARIHARAN



LeetCode Practice

Master Programming Skills with Python · C++ · Rust

2500+

Problems

50+

Companies

10M+

Users

Difficulty Tiers



Easy



Medium



Hard

01. Two Sum

 PUAP01_Python_quick_tutorial.ipynb

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

Q 指令 + 程式碼 + 文字 | ▶ 全部執行 複製到雲端硬碟 連線



```
[ ] 1 instr = input('Please input your score: ')
    2 score = int(instr) #Integer
    3 print(score) # print the value of the variable
    4 print(type(score)) # print the type of the variable
```

Please input your score: 88
88
<class 'int'>

Arithmetic Operators and Expressions Arithmetic Operators

```
+ - * / #Add, subtract, multiply and divide
** #power
// #business
% #remainder
```

```
[ ] 1 print(1 + 2)
    2 print(3 - 4)
```

變數 終端機

<https://leetcode.com/problems/two-sum/description/>
<https://leetcode.cn/problems/two-sum/description/>

66. Plus One

The screenshot shows the LeetCode interface for the problem '66. Plus One'. The left pane contains the problem description, which states: 'You are given a **large integer** represented as an integer array `digits`, where each `digits[i]` is the i^{th} digit of the integer. The digits are ordered from most significant to least significant in left-to-right order. The large integer does not contain any leading 0's. Increment the large integer by one and return *the resulting array of digits*.' It also includes two examples: Example 1 with input `[1,2,3]` and output `[1,2,4]`, and Example 2 with input `[4,3,2,1]` and output `[4,3,2,2]`. The right pane shows a code editor with a Python3 solution:

```
1 class Solution:
2     def plusOne(self, digits: List[int]) -> List[int]:
3 
```

 Below the code editor, there is a message 'You need to log in / sign up to run or submit' and a 'Test Result' section that says 'You must run your code first'.

Problem List < > ↺ Submit

Description Editorial Solutions Submissions

66. Plus One

Easy Topics Companies

You are given a **large integer** represented as an integer array `digits`, where each `digits[i]` is the i^{th} digit of the integer. The digits are ordered from most significant to least significant in left-to-right order. The large integer does not contain any leading 0's.

Increment the large integer by one and return *the resulting array of digits*.

Example 1:

Input: `digits = [1,2,3]`
Output: `[1,2,4]`
Explanation: The array represents the integer 123. Incrementing by one gives $123 + 1 = 124$. Thus, the result should be `[1,2,4]`.

Example 2:

Input: `digits = [4,3,2,1]`
Output: `[4,3,2,2]`

Code

Python3 • Auto

```
1 class Solution:
2     def plusOne(self, digits: List[int]) -> List[int]:
3 
```

Saved

You need to [log in / sign up](#) to run or submit

Testcase Test Result

You must run your code first

<https://leetcode.com/problems/plus-one/description/>

<https://leetcode.cn/problems/plus-one/description/>

20. Valid Parentheses

Description

Editorial

Solutions

Submissions

20. Valid Parentheses

Easy

Topics

Companies

Hint

Given a string `s` containing just the characters `'('`, `)'`, `'{'`, `'}'`, `'['` and `']'`, determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1:

Input: `s = "()"`

Output: `true`

</> Code

Python3 • Auto

```
1 class Solution:
2     def isValid(self, s: str) -> bool:
3
```

Saved

You need to [log in / sign up](#) to run or submit

☒ Testcase

> Test Result

You must run your code first

<https://leetcode.com/problems/valid-parentheses/>
<https://leetcode.cn/problems/valid-parentheses/>

704. Binary Search

Problem List

<

>

🔍

🏠

Submit

📄

🌟

🔧

⚙️

🕒

Register or Log in

Premium

Description

Editorial

Solutions

Submissions

704. Binary Search

EasyTopicsCompanies

Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search `target` in `nums`. If `target` exists, then return its index. Otherwise, return `-1`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [-1,0,3,5,9,12], target = 9`
Output: `4`
Explanation: 9 exists in `nums` and its index is 4

Example 2:

Input: `nums = [-1,0,3,5,9,12], target = 2`
Output: `-1`
Explanation: 2 does not exist in `nums` so return -1

</> Code

Python3 • Auto

1 `class Solution:`
2 `def search(self, nums: List[int], target: int) -> int:`
3

SavedLn 3, Col 9

You need to [log in / sign up](#) to run or submit

☒ Testcase

[> Test Result](#)

You must run your code first

<https://leetcode.com/problems/binary-search/>
<https://leetcode.cn/problems/binary-search/>

53. Maximum Subarray

Problem List

<

>

🔍

🚀

▶

Submit

📄

🌟

🔧

⚙️

🕒

Register or Log in

Premium

Description

Editorial

Solutions

Submissions

53. Maximum Subarray

Medium Topics Companies

Given an integer array `nums`, find the **subarray** with the largest sum, and return *its sum*.

Example 1:

Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]`
Output: 6
Explanation: The subarray `[4,-1,2,1]` has the largest sum 6.

Example 2:

Input: `nums = [1]`
Output: 1
Explanation: The subarray `[1]` has the largest sum 1.

Example 3:

Input: `nums = [5,4,-1,7,8]`
Output: 23
Explanation: The subarray `[5,4,-1,7,8]` has the largest sum 23.

37.8K 519 ☆ 📄 ? 293 Online

</> Code

Python3 • Auto

Ln 1, Col 1

```
1 class Solution:
2     def maxSubArray(self, nums: List[int]) -> int:
3
```

Saved

You need to [log in / sign up](#) to run or submit

✓ Testcase

> Test Result

You must run your code first

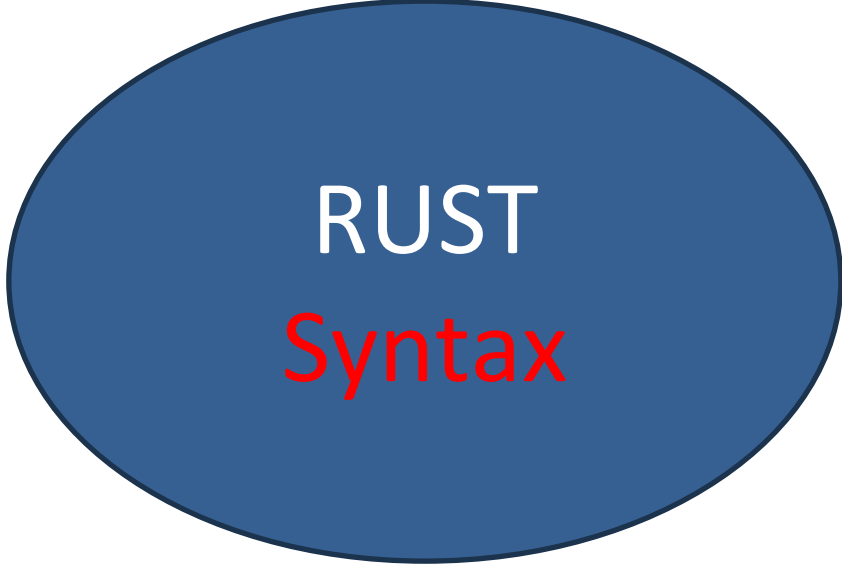
<https://leetcode.com/problems/maximum-subarray/description/>

<https://leetcode.cn/problems/maximum-subarray/description/>

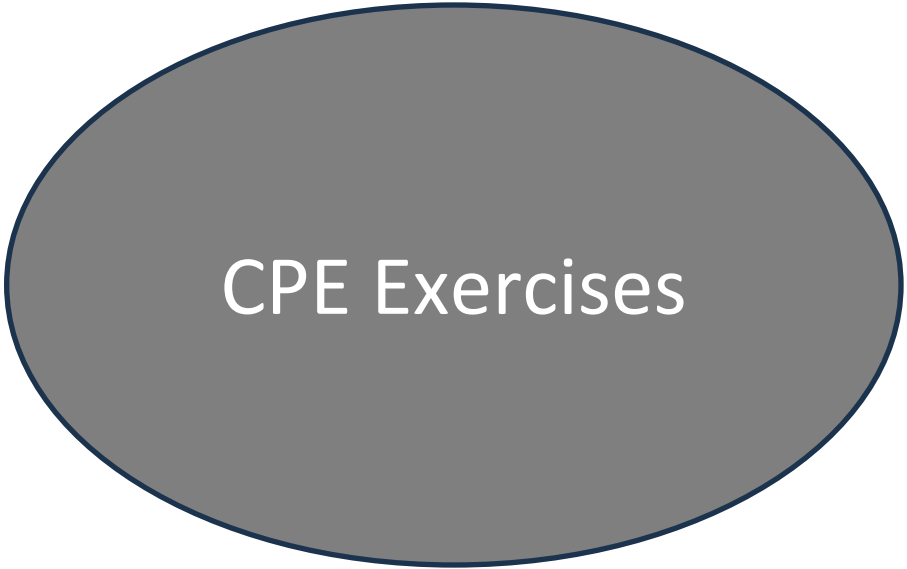
Part-2



Course Activities
Discuss midterm
presentation



RUST
Syntax



CPE Exercises

Introduction to Rust Programming Language

Basic Syntax • Why Rust • Python & C++ User Guides • Cheat Sheets • CodeChef Course



Agenda

01

Why Rust?

Performance, safety, concurrency – the Rust promise

02

Basic Syntax

Variables, types, functions, ownership

03

Python → Rust

Migration guide for Python developers

04

C++ → Rust

Migration guide for C++ developers

05

I/O Cheat Sheet

Input / output comparison: Python, C++, Rust

06

Loop & Conditional Cheat Sheet

for / while / if comparison

07

CodeChef Course

Rust with Problem Solving roadmap overview

Why Rust?

Performance · Memory Safety · Concurrency · Modern Tooling

Why Rust? — The Four Pillars



Blazing Performance

Zero-cost abstractions. No garbage collector.
Performance rivals C and C++.
Direct hardware control.



Memory Safety

No null pointer dereferences.
No buffer overflows.
Ownership system catches bugs at compile time.



Fearless Concurrency

Data races prevented by the type system.
Async/await built-in.
Safe multi-threading without pain.



Modern Tooling

Cargo: world-class build + package manager.
crates.io: 150,000+ libraries.
Excellent IDE support & documentation.

Why Rust? — Industry Adoption & Statistics

#1

Most loved language
9 years in a row (StackOverflow)

2.8×

More energy efficient
than Python (GreenLab study)

70%

Of Microsoft security bugs
would be prevented by Rust

Linux

Kernel now accepts Rust
for new kernel modules (2022)

Who Uses Rust?

- Mozilla — Servo browser engine
- Microsoft — Windows & Azure components
- Mozilla — Servo browser engine
- Google — Android & Chromium OS
- Microsoft — Windows & Azure components
- Google — Android & Chromium OS
- Amazon — AWS Lambda runtime (Firecracker)
- Amazon — AWS Lambda runtime (Firecracker)
- Meta — Mercurial rewrite, internal tools
- Meta — Mercurial rewrite, internal tools
- Cloudflare — Network edge software
- Discord — Replace Go services for performance
- Cloudflare — Network edge software
- Linux Kernel — Driver development
- Discord — Replace Go services for performance
- Linux Kernel — Driver development

Rust Basic Syntax

Variables · Types · Functions · Ownership · Enums

Variables & Data Types

```
// Immutable by default
let x: i32 = 42;
let pi: f64 = 3.14;
let name: &str = "Rust";
let flag: bool = true;

// Mutable
let mut count = 0;
count += 1;

// Constants (always typed)
const MAX_POINTS: u32 = 100_000;

// Shadowing (re-bind with same name)
let x = x + 1;      // x is now 43
let x = x * 2;      // x is now 86
```

💡 *Rust is statically typed — types inferred when possible via let binding*

Primitive Types

Integer	i8, i16, i32, i64, i128, isize u8, u16, u32, u64, u128, usize
Float	f32, f64
Boolean	bool (true / false)
Char	char (Unicode scalar, e.g. '🦀')
String	&str (slice) / String (heap owned)
Tuple	let t = (1, 2.0, 'a'); t.0, t.1, t.2
Array	let a: [i32; 5] = [1,2,3,4,5]; a[0]

Functions & Control Flow

```
// Function: last expression = return value
fn add(a: i32, b: i32) -> i32 {
    a + b          // no semicolon = implicit return
}

// if as expression
let grade = if score >= 90 { "A" } else { "B" };

// loop (infinite, use break to return value)
let mut i = 0;
let result = loop {
    i += 1;
    if i == 5 { break i * 2; }
};

// while
while count < 10 { count += 1; }

// for (range or iterator)
for n in 1..=5 { println!("{}", n); }
for item in &vec { println!("{}", item); }
```

Pattern Matching (match)

```
let num = 7;
match num {
    1          => println!("one"),
    2 | 3      => println!("two or three"),
    4..=6      => println!("four to six"),
    7 if num % 2 != 0
              => println!("odd 7!"),
    _          => println!("other"),
}

// Match with return
let label = match num {
    1..=5 => "small",
    6..=9 => "medium",
    _     => "large",
};
```

Ownership — Rust's Unique Feature

1 Each value has one owner

2 Only one owner at a time

3 When owner leaves scope → value dropped (freed)

```
// MOVE semantics (ownership transferred)
let s1 = String::from("hello");
let s2 = s1;    // s1 is MOVED into s2
// println!("{}", s1); ← compile error!
println!("{}", s2);    // OK

// CLONE (deep copy)
let s3 = s2.clone();
println!("{}", s2, s3); // both valid

// BORROWING (reference, no ownership transfer)
fn print_len(s: &String) -> usize { s.len() }
let s4 = String::from("Rust");
let len = print_len(&s4); // borrow, s4 still owned here
println!("{}", s4, len);

// MUTABLE borrow (only ONE at a time)
fn append(s: &mut String) { s.push_str("!"); }
let mut msg = String::from("Hello");
append(&mut msg);
println!("{}", msg); // "Hello!"
```

Move

Ownership transfers. Original var invalid.
let s2 = s1;

Clone

Deep copy. Both remain valid.
let s2 = s1.clone();

Borrow (&)

Immutable reference. Read-only, any count.
fn f(s: &String)

Borrow (&mut)

Mutable reference. Only 1 at a time.
fn f(s: &mut String)

Structs, Enums & impl

```
// Struct definition
struct Point { x: f64, y: f64 }

// impl block (methods)
impl Point {
    fn new(x: f64, y: f64) -> Point {
        Point { x, y } // shorthand
    }
    fn distance(&self, other: &Point) -> f64 {
        ((self.x-other.x).powi(2)
        +(self.y-other.y).powi(2)).sqrt()
    }
}

let p1 = Point::new(0.0, 0.0);
let p2 = Point::new(3.0, 4.0);
println!("{}", p1.distance(&p2)); // 5.0
```

Enums & Option / Result

```
// Enum with data
enum Shape {
    Circle(f64),
    Rectangle(f64, f64),
}
let area = match shape {
    Shape::Circle(r) => 3.14 * r * r,
    Shape::Rectangle(w, h) => w * h,
};

// Option<T> (replaces null)
let val: Option<i32> = Some(42);
if let Some(v) = val { println!("{}", v); }

// Result<T, E> (replaces exceptions)
use std::fs;
match fs::read_to_string("f.txt") {
    Ok(s) => println!("{}", s),
    Err(e) => println!("Error: {}", e),
}
```


Python → Rust

A Practical Guide for Python Developers

Python Developer's Mental Model Shift

Python

Dynamic typing
type checked at runtime



Rust

Static typing
compiled, inferred by compiler

Python

Garbage collector
auto memory management



Rust

Ownership system
manual-ish, zero-cost, no GC

Python

Exceptions (try/except)
for error handling



Rust

Result<T,E> + Option<T>
no exceptions, explicit errors

Python

Indentation = scope



Rust

Curly braces { } = scope

Python

pip + requirements.txt



Rust

Cargo + Cargo.toml
(much more powerful)

Python

Slow interpreted speed



Rust

Native compiled speed
similar to C

Python → Rust: Side-by-Side Code Comparison

 Python

```
# Variables
x = 42
name = "hello"
pi = 3.14
is_ok = True

# Function
def greet(name: str) -> str:
    return f"Hello, {name}!"

# List & loop
nums = [1, 2, 3, 4, 5]
total = 0
for n in nums:
    total += n

# Dict
d = {"a": 1, "b": 2}
for k, v in d.items():
    print(f"{k}={v}")

# Class
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        print(self.name)
```

 Rust

```
// Variables
let x: i32 = 42;
let name = "hello";
let pi: f64 = 3.14;
let is_ok: bool = true;

// Function
fn greet(name: &str) -> String {
    format!("Hello, {}", name)
}

// Vec & loop
let nums = vec![1, 2, 3, 4, 5];
let total: i32 = nums.iter().sum();

// HashMap
use std::collections::HashMap;
let mut d = HashMap::new();
d.insert("a", 1); d.insert("b", 2);
for (k, v) in &d {
    println!("{}", k, v);
}

// Struct
struct Animal { name: String }
impl Animal {
    fn speak(&self) { println!("{}", self.name); }
}
```


C++ → Rust

A Practical Guide for C++ Developers

C++ Developer's Mental Model Shift

Memory Management

C++

Manual new/delete
or smart pointers



Rust

Ownership system
auto drop, no leak/UAF

Null Pointers

C++

nullptr risks
segfault at runtime



Rust

Option<T>
None is explicit, safe

Undefined Behavior

C++

Many UB cases
leak through compiler



Rust

UB only in unsafe {}
bounds checked by default

Error Handling

C++

Exceptions (throw/catch)
or error codes



Rust

Result<T,E> + ?
no exception overhead

Templates vs Generics

C++

Templates (compile-time
duck typing)



Rust

Generics + Traits
explicit, bounded types

Header Files

C++

#include, .h/.cpp split
forwards declarations



Rust

No headers needed
mod system + Cargo

C++ → Rust: Side-by-Side Code Comparison



C++

```
// Variables & types
int x = 42;
std::string name = "hello";
const double PI = 3.14;
bool ok = true;

// Function
std::string greet(const std::string& n) {
    return "Hello, " + n + "!";
}

// Vector & loop
std::vector<int> nums = {1,2,3,4,5};
int total = 0;
for (int n : nums) total += n;

// Pointer / smart pointer
auto ptr = std::make_unique<int>(42);
std::cout << *ptr << std::endl;

// Class
class Animal {
    std::string name;
public:
    Animal(std::string n) : name(n) {}
    void speak() { std::cout << name; }
};
```



Rust

```
// Variables & types
let x: i32 = 42;
let name = String::from("hello");
const PI: f64 = 3.14;
let ok: bool = true;

// Function
fn greet(n: &str) -> String {
    format!("Hello, {}", n)
}

// Vec & loop
let nums = vec![1, 2, 3, 4, 5];
let total: i32 = nums.iter().sum();

// Box<T> (heap allocation, like unique_ptr)
let ptr = Box::new(42);
println!("{}", *ptr);

// Struct + impl
struct Animal { name: String }
impl Animal {
    fn new(n: &str) -> Animal {
        Animal { name: n.to_string() }
    }
    fn speak(&self) { println!("{}", self.name); }
}
```

Cheat Sheets

I/O • Loops • Conditionals — Python vs C++ vs Rust

Cheat Sheet: Basic Input / Output			
Operation	 Python	 C++	 Rust
Print string	<code>print("Hello")</code>	<code>cout << "Hello";</code>	<code>println!("Hello");</code>
Print var	<code>print(x)</code>	<code>cout << x;</code>	<code>println!("{}", x);</code>
Print fmt	<code>print(f"{a} + {b}")</code>	<code>cout << a << "+" << b;</code>	<code>println!("{}", a, b);</code>
Newline	<code>print("\n")</code> or <code>println</code>	<code>cout << endl;</code> or <code>"\n"</code>	<code>println!()</code> adds <code>\n</code> auto
Read string	<code>s = input("Prompt: ")</code>	<code>cin >> s;</code>	<code>use std::io;</code> <code>io::stdin().read_line(&mut s);</code>
Read int	<code>n = int(input())</code>	<code>cin >> n;</code>	<code>let n: i32 = s.trim().parse().unwrap();</code>
Read line	<code>s = input()</code>	<code>getline(cin, s);</code>	<code>io::stdin().read_line(&mut s).unwrap();</code>
Print to stderr	<code>import sys</code> <code>sys.stderr.write()</code>	<code>cerr << "err";</code>	<code>eprintln!("err {}", e);</code>

Cheat Sheet: Loops (for / while / loop)			
Type	 Python	 C++	 Rust
for range	<pre>for i in range(5): print(i)</pre>	<pre>for(int i=0;i<5;i++) cout<<i;</pre>	<pre>for i in 0..5 { println!("{}",i); }</pre>
for incl.	<pre>for i in range(1,6): print(i)</pre>	<pre>for(int i=1;i<=5;i++) cout<<i;</pre>	<pre>for i in 1..=5 { println!("{}",i); }</pre>
for iter	<pre>for x in my_list: print(x)</pre>	<pre>for(auto& x : v) cout<<x;</pre>	<pre>for x in &vec { println!("{}",x); }</pre>
while	<pre>while cond: do_stuff()</pre>	<pre>while(cond) { do_stuff(); }</pre>	<pre>while cond { do_stuff(); }</pre>
do-while	<pre>while True: f() if not c: break</pre>	<pre>do { f(); } while(c);</pre>	<pre>loop { f(); if !c { break; } }</pre>
infinite	<pre>while True: f()</pre>	<pre>while(true) { f(); }</pre>	<pre>loop { f(); }</pre>
break/cont.	<pre>break / continue</pre>	<pre>break; / continue;</pre>	<pre>break; / continue;</pre>
enumerate	<pre>for i,v in enumerate(lst): ...</pre>	<pre>for(int i=0;i<n;i++) v=vec[i]; ...</pre>	<pre>for (i,v) in v.iter() .enumerate() { ... }</pre>

Cheat Sheet: Conditionals (if / else / match)			
Type	 Python	 C++	 Rust
if / else	<pre>if cond: f() else: g()</pre>	<pre>if(cond){ f(); }else{ g();}</pre>	<pre>if cond { f(); } else { g(); }</pre>
else if	<pre>elif cond2: h()</pre>	<pre>else if(cond2){ h(); }</pre>	<pre>} else if cond2 { h(); }</pre>
ternary	<pre>x = a if cond else b</pre>	<pre>x = cond ? a : b;</pre>	<pre>let x = if cond { a } else { b };</pre>
switch/match	<pre>match x: case 1: f() case 2: g()</pre>	<pre>switch(x){ case 1: f(); break; case 2: g();}</pre>	<pre>match x { 1 => f(), 2 => g(), _ => h(), }</pre>
range match	N/A (use if/elif)	N/A (use if/else)	<pre>match n { 1..=5 => low(), _ => high(), }</pre>
guard match	N/A	N/A	<pre>match n { x if x<0 => neg(x), _ => pos() }</pre>
if let	N/A (use isinstance)	N/A (use std::get)	<pre>if let Some(v) = opt { use(v); }</pre>
short-circuit	<pre>a and b / a or b</pre>	<pre>a && b / a b</pre>	<pre>a && b / a b</pre>

CodeChef Course

Rust with Problem Solving — codechef.com/roadmap/rust-dsa

CodeChef: Rust with Problem Solving — Overview



Rust with Problem Solving



codechef.com/roadmap/rust-dsa

Learn the basics of Rust and problem solving. Use practice modules to boost your logical thinking.

Structured Learning



Step-by-step modules from Rust syntax basics to DSA problem solving with guided exercises.

In-Browser IDE



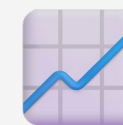
Write and run Rust code directly in the browser. No local setup required to start learning.

500+ Problems



Curated hand-picked problems progressing from easy warmups to intermediate DSA challenges.

Beginner Friendly



No prior Rust knowledge needed. Suitable for Python/C++ programmers transitioning to Rust.

CodeChef Rust Course — Learning Modules

Phase 1: Rust Foundations

1 Setup & Hello World — Cargo, rustc, first program

2 Variables, Types & Mutability

3 Functions, Control Flow (if / loop / for / while)

4 Ownership, Borrowing & References

5 Structs, Enums & Pattern Matching

Phase 2: Problem Solving with Rust

1 I/O Handling — stdin/stdout for competitive programming

2 Arrays, Vectors & Strings

3 HashMap, HashSet & Collections

4 Sorting & Searching Algorithms

5 Recursion & Basic Algorithms

Phase 3: Data Structures

1 Linear DS: Stack, Queue, Linked List in Rust

2 Binary Search & Two Pointers

3 Trees & Graphs (BFS / DFS)

4 Dynamic Programming fundamentals

5 Practice: 500+ curated contest problems

Getting Started — Rust with Problem Solving

Step-by-Step Guide to Enroll & Start



Step 1: Visit CodeChef

Go to codechef.com/roadmap/rust-dsa in your browser



Step 2: Create Account

Sign up for free at codechef.com (or log in if you have an account)



Step 3: Enrol in Course

Click 'Enrol Now' on the Rust with Problem Solving roadmap page



Step 4: Start Module 1

Begin with Rust basics: variables, types, I/O.
No local setup needed.



Step 5: Solve Problems

After each lesson, solve 5+ practice problems to reinforce concepts



Step 6: Level Up

Progress through all 3 phases: foundations
→ problem solving → DSA

Summary & Resources

Key Takeaways

Rust = Performance + Safety + Concurrency

Ownership eliminates entire classes of bugs

Cargo makes dependency management easy

Rich type system catches errors at compile time

Community loves Rust — 9 years #1 most loved

Learning Resources

 **Official Rust Book (free)**

doc.rust-lang.org/book

 **Rustlings — interactive**

github.com/rust-lang/rustlings

 **Rust by Example**

doc.rust-lang.org/rust-by-example

 **CodeChef Rust Course**

codechef.com/roadmap/rust-dsa

 **crates.io — packages**

crates.io

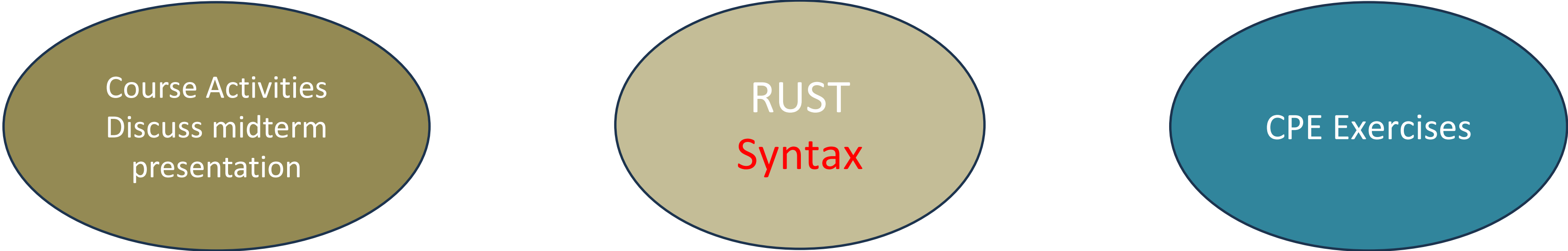
 **Rust Playground (online IDE)**

play.rust-lang.org



Start your Rust journey today!

Part-3



Course Activities
Discuss midterm
presentation

RUST
Syntax

CPE Exercises

CodeChef: Rust with Problem Solving — Overview

CPE Exercise@PU

Home

Problems

Contests

Status

Rank

About

root

202605CPE Exercise-01

-14:31:54

202605CPE Exercise-01

uva10591 : Happy Number

uva10079 : Pizza Cutting

uva100 : The 3n + 1 problem

uva11063 : B2-sequence

StartAt	EndAt	ContestType	Rule	Creator
2026-4-21 15:00:00	2026-4-27 21:00:00	Public	ACM	htchu2

Overview

Announcements

Problems

Submissions

Rankings

Admin Helper

Online Judge Footer (靜宜大學CPE練習網站)

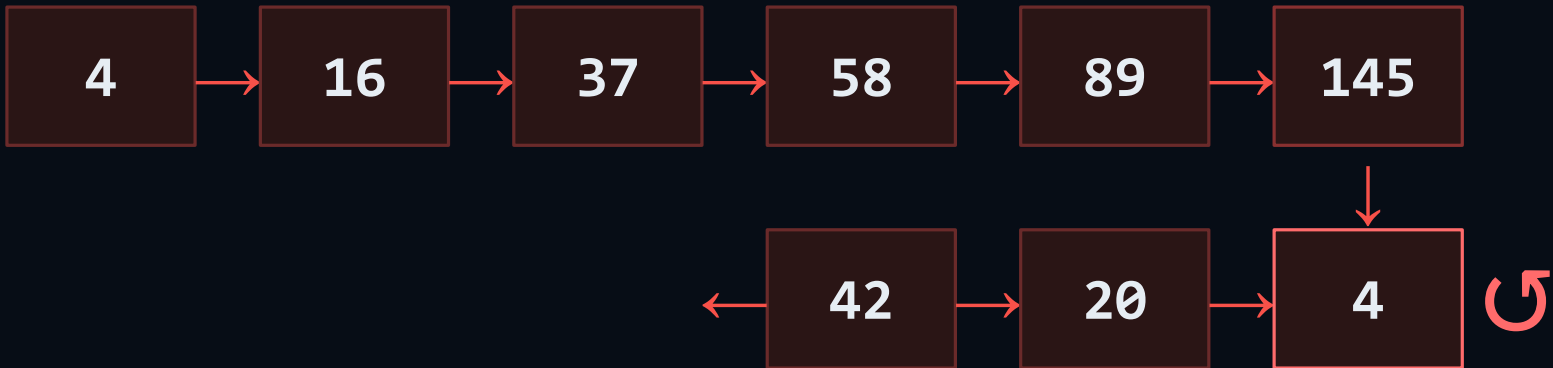
Powered by OnlineJudge Version: 20210928-acce7

Step-by-Step Digit² Sum Trace

✓ **HAPPY: n = 7** (reaches 1)



✗ **UNHAPPY: n = 4** (infinite cycle)



Known 8-step unhappy cycle:
4→16→37→58→89→145→42→20→4→...

Algorithm: use set<int> to detect revisited values.

- $n==1$ reached → Happy ✓ | n seen before → Unhappy ✗

PROBLEM STATEMENT

A number is HAPPY if repeatedly replacing it with the sum of squares of its digits eventually produces 1. If it loops endlessly → UNHAPPY.

Given T numbers, classify each one.

- Constraints: $1 \leq T \leq 100$, $1 \leq N \leq 10^9$

KEY INSIGHT

Use set<int> to detect revisited values.

- Reach 1 → Happy ✓
- See same value again → Unhappy ✗

All unhappy numbers cycle through the same 8-node cycle: {4,16,37,58,89,145,42,20}

SAMPLE INPUT / OUTPUT

Input	Output
1	1 is a Happy Number!
7	7 is a Happy Number!
4	4 is an Unhappy Number!
100	100 is a Happy Number!

Maximum Pieces from n Straight Cuts



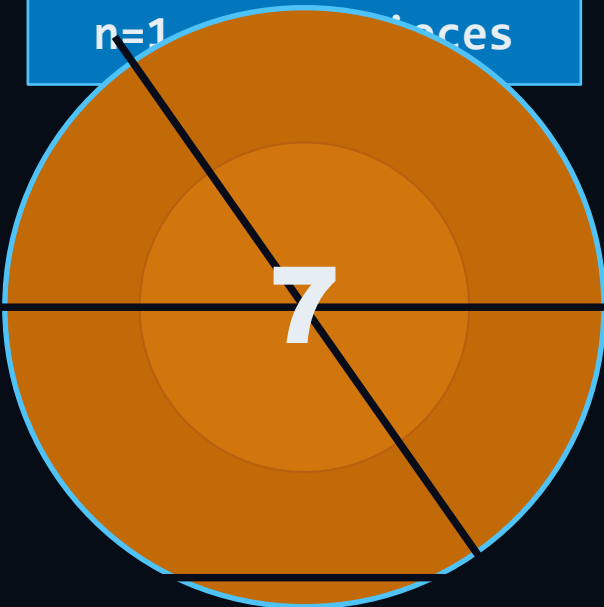
n=0 → 1 piece



n=1 → 2 pieces



n=2 → 4 pieces



n=3 → 7 pieces

Difference pattern: +1 +2 +3 +4 ... +n
P(n) = $n \times (n+1) / 2 + 1$ ← use long long!

PROBLEM STATEMENT

A circular pizza is cut with n straight lines.
Find the MAXIMUM number of pieces possible.

Each cut must cross ALL previous cuts at distinct interior points (no parallel cuts, no three cuts meeting at one point).
• Constraints: $0 \leq n \leq 10^{15}$

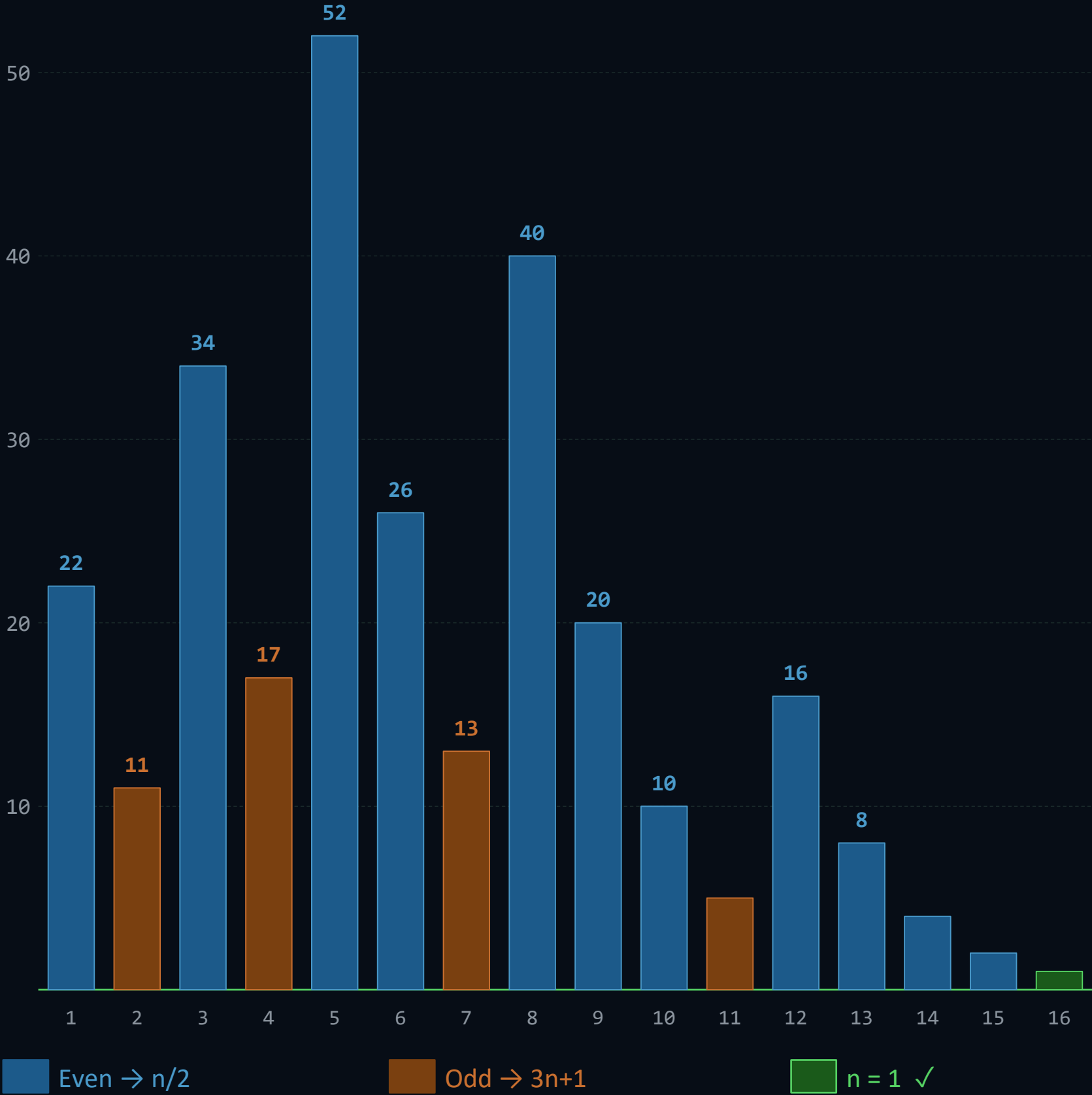
KEY INSIGHT

Closed form: $P(n) = n \times (n+1) / 2 + 1$
The k-th cut intersects k-1 prior cuts, creating exactly k new pieces.
⚠ n up to 10^{15} → MUST use long long!
 $n \times (n+1)$ overflows 32-bit integer.

SAMPLE INPUT / OUTPUT

Input	Output
0	1
1	2
2	4
3	7

Collatz Sequence n = 22 (cycle length = 16)



PROBLEM STATEMENT

The Collatz (3n+1) conjecture:

- If n is even: $n = n / 2$
- If n is odd: $n = 3n + 1$

Repeat until $n = 1$. Count = "cycle length".

Given i and j, find MAX cycle length in [i,j].
Note: i may be greater than j!

KEY INSIGHT

- ① Swap i,j so $lo = \min(i,j)$, $hi = \max(i,j)$!
- ② Intermediate values exceed int → long long
- ③ Memoize: `map<long long,int>` for speed
- ④ Output: "i j maxLen" (original i,j order)

SAMPLE INPUT / OUTPUT

Input	Output
1 10	1 10 20
100 200	100 200 125
201 210	201 210 89
900 1000	900 1000 174

Pairwise Sum Matrix ($b_i + b_j, i \leq j$)

[1, 2, 3, 5]					[1, 2, 3, 4]				
	1	2	3	5		1	2	3	4
1	2	3	4	6	1	2	3	4	5
2	.	4	5	7	2	.	4	5	6
3	.	.	6	8	3	.	.	6	7
5	.	.	.	11	4	.	.	.	8
✓ Valid B2-Sequence					✗ Not B2-Sequence				

RED = duplicate sum (violates B2) | GREEN/PURPLE = unique sum | . = $i > j$ (skipped)

PROBLEM STATEMENT

A B2-sequence: strictly increasing positive integers $b_1 < b_2 < \dots < b_n$ where ALL pairwise sums $b_i + b_j$ ($i \leq j$) are DISTINCT.
Self-sums $2 \cdot b_i$ are included ($i=j$ is allowed).

Determine if a given sequence is B2.
Output "Case #X: It is/is not a B2-sequence."

KEY INSIGHT

- Two checks:
- ① Strictly increasing & all ≥ 1
 - ② Insert each $b_i + b_j$ into `set<long long>`; if already present $\rightarrow$ NOT B2-sequence.
- Total pairs = $n \cdot (n+1) / 2$ (upper triangle).

SAMPLE INPUT / OUTPUT

Input	Output
4 1 2 3 5	It is a B2-sequence.
3 1 2 3	It is not a B2-sequence.
4 1 2 3 4	It is not a B2-sequence.
1 5	It is a B2-sequence.